

## Lezione 7 – Socket Java

Reti di calcolatori

Modulo 4 - Tecniche di programmazione distribuita

Unità didattica 1 – Socket Library

**Ernesto Damiani**

---

Università degli Studi di Milano - Ssri - CDL ONLINE

### Cos' è un socket?

- Socket
  - La combinazione di un indirizzo IP e di un numero di porta (RFC 793, specificazione originale TCP)
  - Il nome delle API (Application Programming Interface) derivate da Berkeley per applicazioni usando i protocolli TCP/IP
  - Due tipi
    - **Socket di flusso**: flussi affidabili di comunicazione connessi two-way
    - **Socket di datagram**
- Coppia di socket
  - Specificano i due punti finali che identificano univocamente una connessione TCP full duplex
  - quadrupla: (indirizzo IP del client, numero della porta del client, indirizzo IP del server, numero della porta del server)

## Applicazioni client-server

- Implementazione di uno standard definito in una RFC (FTP, HTTP, SMTP...)
  - Conforme alle regole imposte dalla RFC
  - Si deve usare il numero di porta associato al protocollo
  - I numeri di porta noti sono gestiti da IANA (Internet Assigned Numbers Authority)
- Applicazione proprietaria client-server
  - Un singolo sviluppatore (o un team) crea sia il programma client sia il programma server
  - Lo sviluppatore ha il controllo completo
  - Non si deve usare uno dei numeri di porta noti definiti nelle RFC

## Socket per server e client

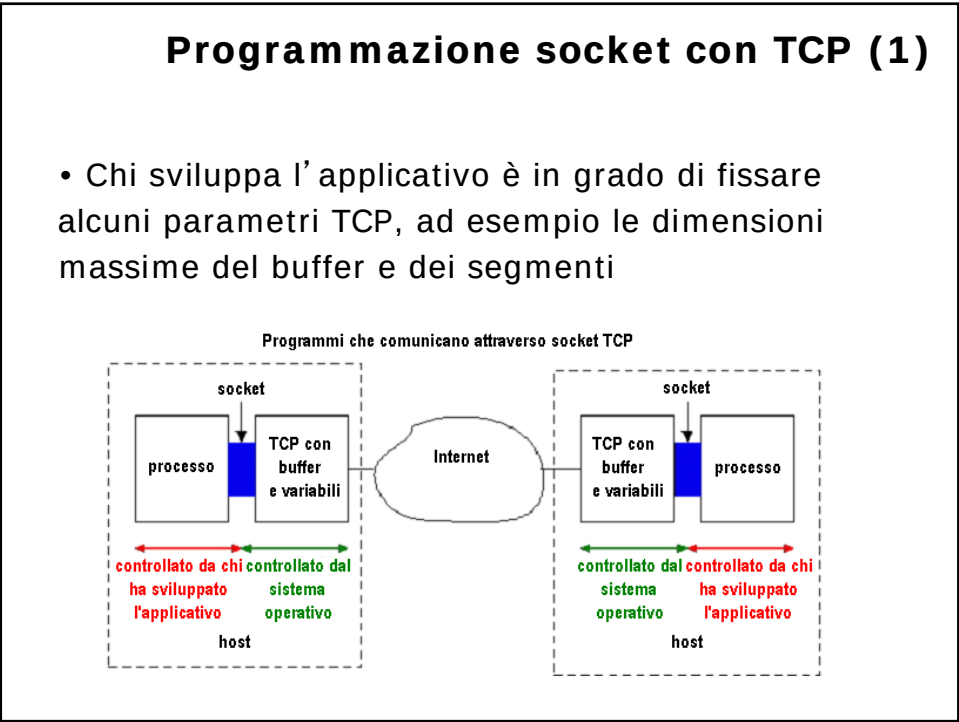
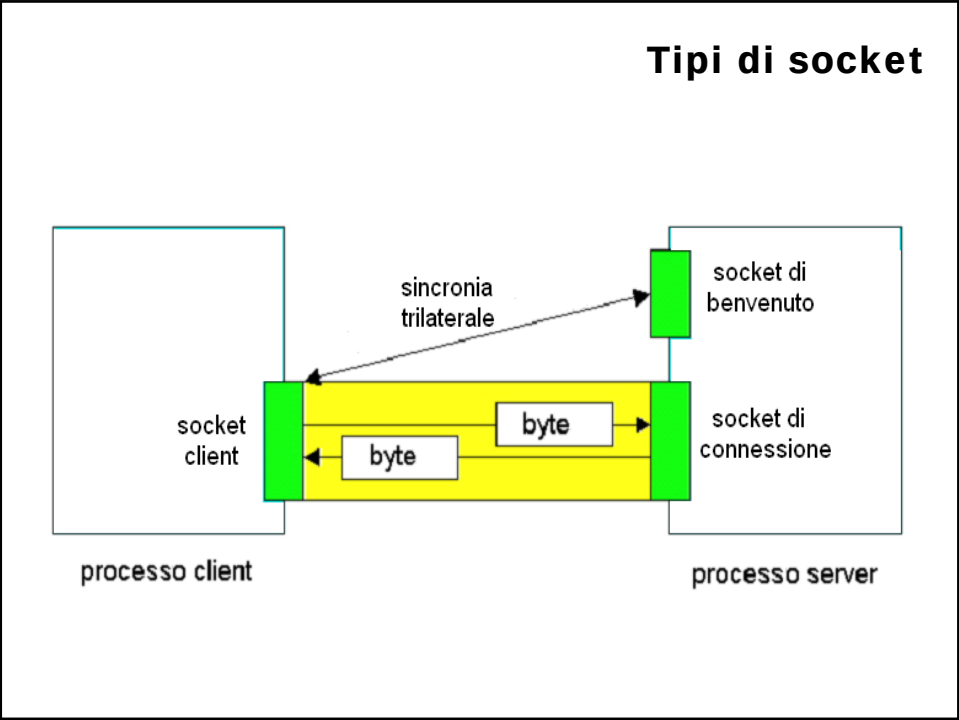
- Server
  - **Socket di benvenuto**
    - Dà il benvenuto a un contatto iniziale da un client
  - **Socket di connessione**
    - È creato dopo il contatto iniziale del client
    - Il nuovo socket che è dedicato al particolare client
- Client
  - **Socket client**
    - una connessione TCP al server viene iniziata creando un oggetto socket (sincronia trilaterale )
    - Va specificato l'indirizzo del processo server, cioè l'indirizzo IP del server e il numero di porta del processo

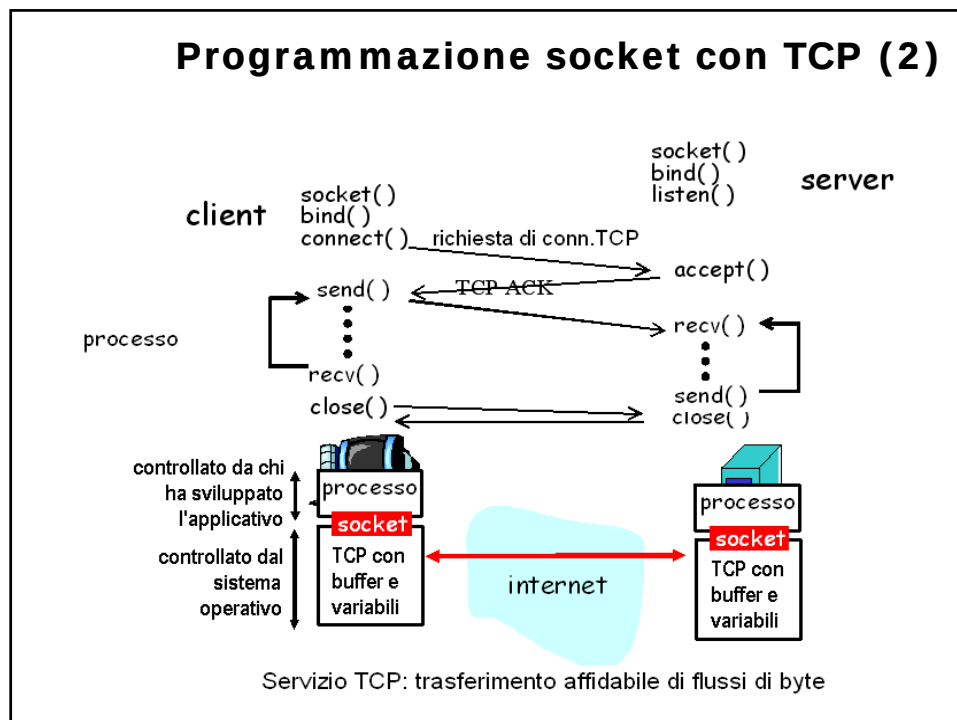
### Chiamate socket funzionali (1)

- **socket ()**: crea un socket
- **bind()**: associa un socket a un indirizzo IP locale e a una porta #
- **listen()**: aspetta passivamente le connessioni
- **connect()**: avvia una connessione a un altro socket
- **accept()**: accetta una nuova connessione

### Chiamate socket funzionali (2)

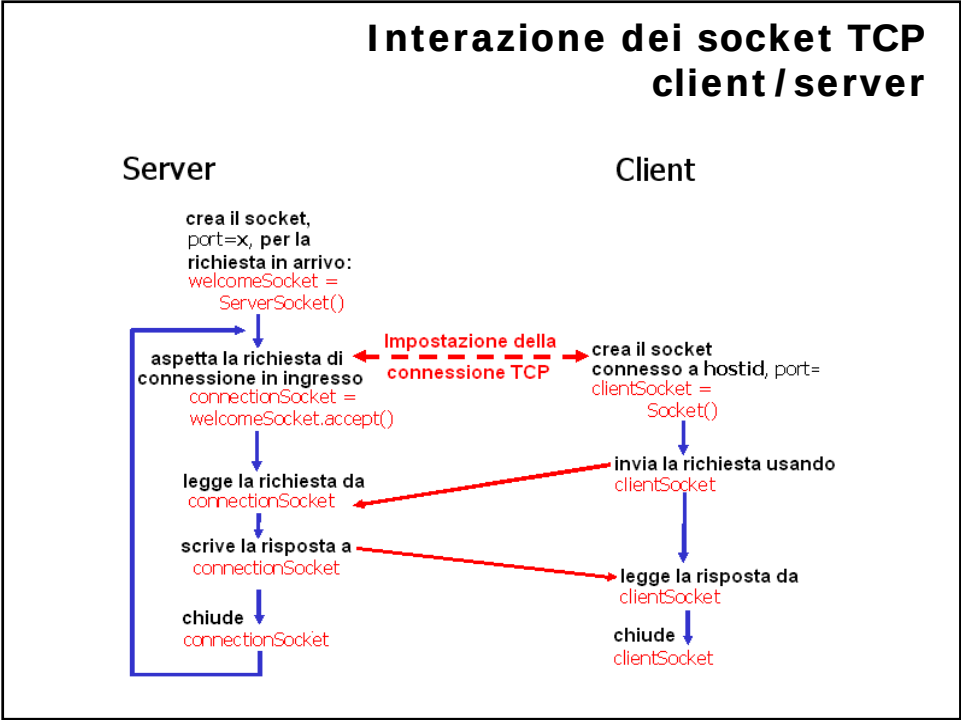
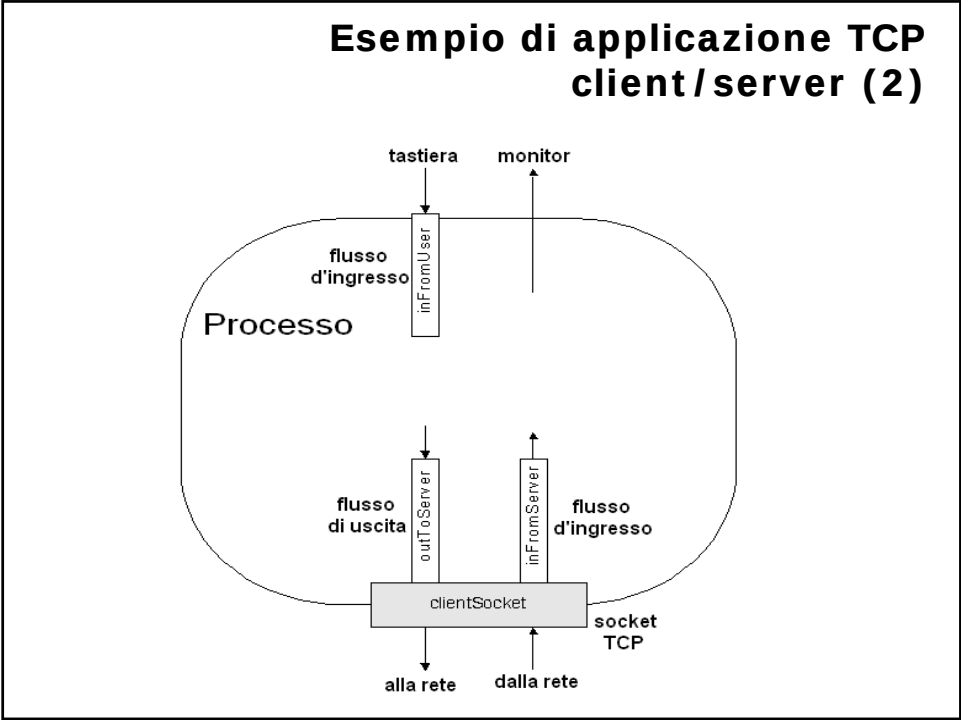
- **Write()**: scrive i dati su un socket
- **Read()**: legge i dati da un socket
- **sendto()**: invia un datagram a un altro socket UDP
- **recvfrom()**: legge un datagram da un socket UDP
- **close()**: chiude un socket (fa crollare la connessione)





### Esempio di applicazione TCP client / server (1)

- Il client legge la riga dall'input standard (flusso **inFromUser**), e la invia al server tramite socket (flusso **outToServer**)
- Il server legge la riga dal socket
- Il server converte la riga in lettere maiuscole e la invia al client
- Il client legge dal socket la riga modificata e la visualizza (flusso **inFromServer**)



## Socket TCP Java (1)

### Il Package java.net

- java.net.Socket
  - Implementa i socket client (chiamati anche solo “socket”)
  - Un endpoint per la comunicazione tra due macchine
  - Costruttore e metodi
    - Socket(String host, int port): Crea un socket di flussi e lo connette al numero di porta specificato sull’ host con nome
    - InputStream getInputStream()
    - OutputStream getOutputStream()
    - close()

## Socket TCP Java (2)

- java.net.ServerSocket
  - Implementa socket server
  - Aspetta che arrivino le richieste sulla rete
  - Esegue alcune operazioni in base alla richiesta
  - Costruttore e metodi
    - ServerSocket(int port)
    - Socket Accept(): controlla che avvenga la connessione a questo socket e la accetta (Questo metodo è bloccato fin che non viene eseguita una connessione)

### TCPClient.java (1)

```
import java.io.*;
import java.net.*;

class TCPClient {
    public static void main(String argv[]) throws Exception
    {
        String sentence;
        String modifiedSentence;

        BufferedReader inFromUser =
            new BufferedReader(new InputStreamReader(System.in));

        Socket clientSocket = new Socket("hostname", 6789);

        DataOutputStream outToServer =
            new DataOutputStream(clientSocket.getOutputStream());
```

### TCPClient.java (1)

```
        BufferedReader inFromServer =
            new BufferedReader(new
                InputStreamReader(clientSocket.getInputStream()));

        sentence = inFromUser.readLine();

        outToServer.writeBytes(sentence + '\n');

        modifiedSentence = inFromServer.readLine();

        System.out.println("FROM SERVER: " + modifiedSentence);

        clientSocket.close();
    }
}
```



### TCPServer.java (1)

```
import java.io.*;
import java.net.*;
class TCPServer {
    public static void main(String argv[]) throws Exception
    {
        String clientSentence;
        String capitalizedSentence;

        ServerSocket welcomeSocket = new ServerSocket(6789);

        while(true) {

            Socket connectionSocket = welcomeSocket.accept();

            BufferedReader inFromClient = new BufferedReader(new
                InputStreamReader(connectionSocket.getInputStream()));
```

### TCPServer.java (2)

```
        DataOutputStream outToClient =
            new DataOutputStream(connectionSocket.getOutputStream());

        clientSentence = inFromClient.readLine();

        capitalizedSentence = clientSentence.toUpperCase() + '\n';

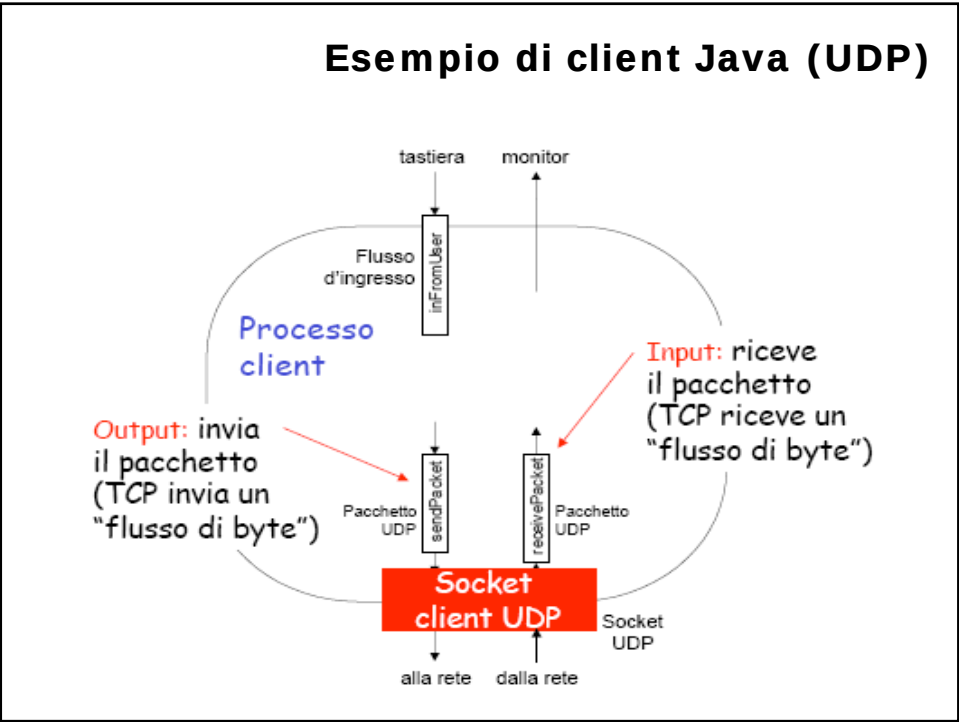
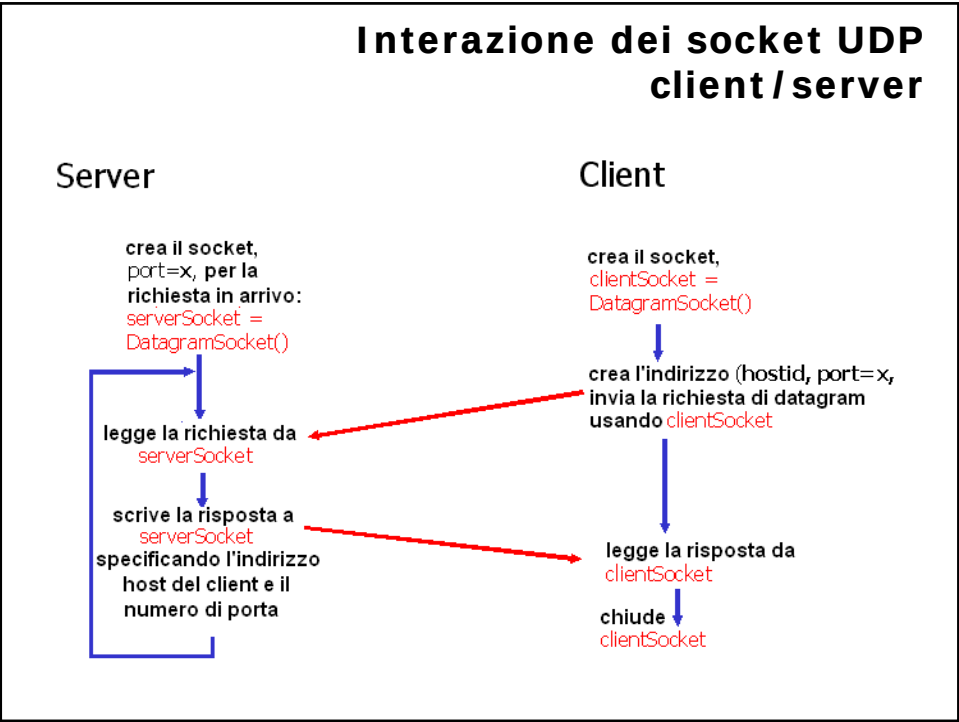
        outToClient.writeBytes(capitalizedSentence);
    }
}
```

## **UDP**

- Servizio non affidabile e senza connessioni
- Non c'è una fase handshaking iniziale
- Non ha un pipe
- I dati trasmessi possono perdersi o arrivare a destinazione in un ordine diverso da quello d'invio

## **Programmazione socket con UDP**

- Non serve un socket di benvenuto
- Non c'è alcun flusso allegato ai socket
- L'host mittente crea "pacchetti" allegando l'indirizzo IP di destinazione e il numero di porta a ogni batch di byte
- Il processo destinatario deve estrarre dal pacchetto ricevuto i byte d'informazione



## Socket UDP JAVA

### Il Package java.net

- java.net.DatagramSocket
  - Un socket per inviare e ricevere i pacchetti di datagram
  - Costruttore e metodi
    - DatagramSocket(int port): Costruisce un socket di datagram e lo lega alla porta specificata sulla macchina host locale
    - void receive( DatagramPacket p)
    - void send( DatagramPacket p)
    - void close()

### UDPClient.java (1)

```
import java.io.*;
import java.net.*;

class UDPClient {
    public static void main(String args[]) throws Exception
    {
        BufferedReader inFromUser =
            new BufferedReader(new InputStreamReader(System.in));

        DatagramSocket clientSocket = new DatagramSocket();

        InetAddress IPAddress =
InetAddress.getByName("hostname");

        byte[] sendData = new byte[1024];
        byte[] receiveData = new byte[1024];

        String sentence = inFromUser.readLine();

        sendData = sentence.getBytes();
    }
}
```

### UDPClient.java (2)

```
DatagramPacket sendPacket =  
    new DatagramPacket(sendData, sendData.length,  
    IPAddress, 9876);  
clientSocket.send(sendPacket);  
  
DatagramPacket receivePacket =  
    new DatagramPacket(receiveData, receiveData.length);  
  
clientSocket.receive(receivePacket);  
  
String modifiedSentence =  
    new String(receivePacket.getData());  
  
System.out.println("FROM SERVER:" + modifiedSentence);  
  
clientSocket.close();  
  
    }  
}
```

### UDPServer.java (1)

```
import java.io.*;  
import java.net.*;  
  
class UDPServer {  
    public static void main(String args[]) throws Exception  
    {  
  
        DatagramSocket serverSocket = new  
        DatagramSocket(9876);  
  
        byte[] receiveData = new byte[1024];  
        byte[] sendData = new byte[1024];  
  
        while(true)  
        {  
  
            DatagramPacket receivePacket =  
                new DatagramPacket(receiveData, receiveData.length);  
  
            serverSocket.receive(receivePacket);  
  
            String sentence = new String(receivePacket.getData());  

```

## UDPServer.java (2)

```
InetAddress IPAddress = receivePacket.getAddress();  
int port = receivePacket.getPort();  
String capitalizedSentence = sentence.toUpperCase();  
sendData = capitalizedSentence.getBytes();  
DatagramPacket sendPacket =  
    new DatagramPacket(sendData, sendData.length, IPAddress, port);  
serverSocket.send(sendPacket);  
  
    }  
}  
}
```

